Automated Reconciliation and Testing (ART) framework

Document Control

Document Status

Date	Version	Status
17-Sep-2025	v.235	ENDORSED

Document Authors

Name	Role
Naresh Kala	Platform Team Member
Koustubh Kasalkar	Offshore Lead
Syed Fayaz	Onshore Lead

Reviewers and Approvers

Name	Role	Review Section	Jira Ticket /Status	Version

Yi-Jing Chung	Lead Engineer -Data		APU-943 - Design Document : Review and Sign-off recon framework	0.6
Paul Dudding	Platform Architect -Data		APU-945 - Design Document : Review and Sign-off recon framework	0.6
Miditha Fernando	Lead Engineer -Data		APU-944 - Design Document : Review and Sign-off recon framework	0.6
Daniel Dove	Lead Engineer -Data		APU-1164 - Design Document : Review and Sign-off recon framework	0.6

Endorsers and Approvers

Name	Role	Purpose	Version	Date	Status
Amy Di Benedetto		Endorse Proposed Design	0.6		APU-946 - Design Document : Review and Sign-off recon framework
Grace Shin		Endorse Proposed Design	0.6		APU-1084 - Design Document : Review and Sign-off recon framework

Key Revision History

Version	Date	Description	Author (s)	Status
0.1	14/08 /2025	First Draft	Naresh Kala	Draft
0.2	08/09 /2025	Added details based on inline comments	Fayaz Syed	Draft
0.3	08/09 /2025	Added the additional fields in the Metadata_Test_cases,Test_Results Incorporating the design feedback and new requirements suggested	Naresh Kala	Draft
0.4	09/09 /2025	Added details based om inline comments	Naresh Kala	Draft
0.5	12/09 /2025	Incorporated the review comments	Naresh Kala	Draft
0.6	15/09 /2025	Incorporated the review comments 1) LAST_FILE_PROCESSED_DATE (rename existing column LAST_PROCESS_DATE) 2) Add new column DATA_DATE (where DATA_DATE variable is passed) add "explain compiled section" for data_date test case sample	Naresh Kala	Draft
0.7	28/09 /2025	Mandatory Columns – `METADATA_TEST_CASES Table` tolerance_percentage (optional): Acceptable variance threshold (default: 1%) Updated the core macros Parameters	Pavan Jangam	Draft

Introduction

Purpose

The 'Automated Reconciliation and Testing (ART) framework' focuses on data reconciliation between two datasets, usually, source(s) set and target set, based on specific comparison logic defined as test cases. Based on these rules, reconciliation is performed to validate data between these two datasets and produce results that indicate binary results: failure or success.

There are two components to the recon framework

1) Source Landing (Scope is clarified in section 'Scope' below)

2) Landing Zones (Raw/Foundation/Curated)

Reconciliation is identified as a Governance by Design initiated change and hence being prioritized.

The relevant pages that refer the Governance by Design controls are

1) DRC-09 Data reconciliation after transport

2) DRC-10 Data reconciliation after processing or enrichment

Scope

The data reconciliation framework is designed for validating data movement from source to landing tables, and from landing to raw, foundation, and curated layer tables.

Kafka source data and Fivetran-based data reconciliation are out of scope for this framework, as they require wider enterprise decision and agreement.

> **Note on dependecy**
>
> The Source Landing reconciliation is dependent on source system providing additional manifest files in addition to the data file. This is being prioritized and worked through with Source System teams and WLA. A high level design guidance is documented in DRAFT - Reconciliation with Source Systems. The final design will be determined once Landing team and Source System Solution Designers complete the design. Current SourceLanding Recon Framework is based off these guidelines, and in case this changes the framework would need to be modified.

The data reconciliation DAG runs independently after the data loading process as a post-load validation step. It is scheduled to execute at a defined frequency (daily, weekly, or monthly) and is designed to complete without failure. The results of the test cases are stored in a final results table.

For now, any recon test case mismatches will be verified manually after integration is complete. In the future, monitoring and alerting — including email notifications via Splunk — will be implemented as part of the automated process.

Scope Includes :

- Verifying row counts and aggregated totals, data freshness, uniqueness between source and target tables.
- Comparing record-level data to detect mismatches or data loss.
- Executed as a post-load validation, scheduled independently (e.g., daily, weekly,monthly).

Note:
This document is only about the data reconciliation process, which verifies data loaded is correct from source to target after ingestion.
It does not include dbt tests or model contract related tests, which are separate and execute during dbt models execution to validate data quality.
There is no overlap between data reconciliation and dbt tests — dbt tests are handled separately within the dbt zone project. A mismatch in reconciliation will not hold the dbt model pipeline execute it

The tables below give a clear gives which test cases are handled by **dbt tests** and which ones are covered through **Data Reconciliation**.

S.NO	In-built dbt tests	DATA RECONCILIATION (ART Framework)
1	Unique_Columns	Row Counts
2	Not_Null	Aggregated Totals
3	Accepted_Values	Data Freshness (beyond Sources only)
4	Relationships	Uniqueness

Out of Scope

- Kafka Source based data reconciliation
- Five Tran-based data reconciliation

Overview of Data Reconciliation

This is an automated, metadata-driven framework designed for test automation and data reconciliation across data pipeline layers.

Table-to-Table Comparison

Applicable for Landing Raw Foundation Curated

DRC-10 Data reconciliation after processing or enrichment - Digital, Data & Analytics - BNZ Confluence

- Row Count Validation– Ensures consistency in record volume between source and target tables
- Aggregation Validation – Verifies accuracy of aggregated metrics across layers
- Uniqueness Validation-Confirms presence of unique keys or values where required
- Data Freshness Validation– Checks that data is up-to-date and timely across stages

Source Manifest to Landing Comparison

Applicable for Source Landing

DRC-09 Data reconciliation after transport - Digital, Data & Analytics - BNZ Confluence

- Record Count Validation – Validates that ingested records match source expectations
- Hash Checksum Validation – Confirms data integrity by comparing hash values between source and landing

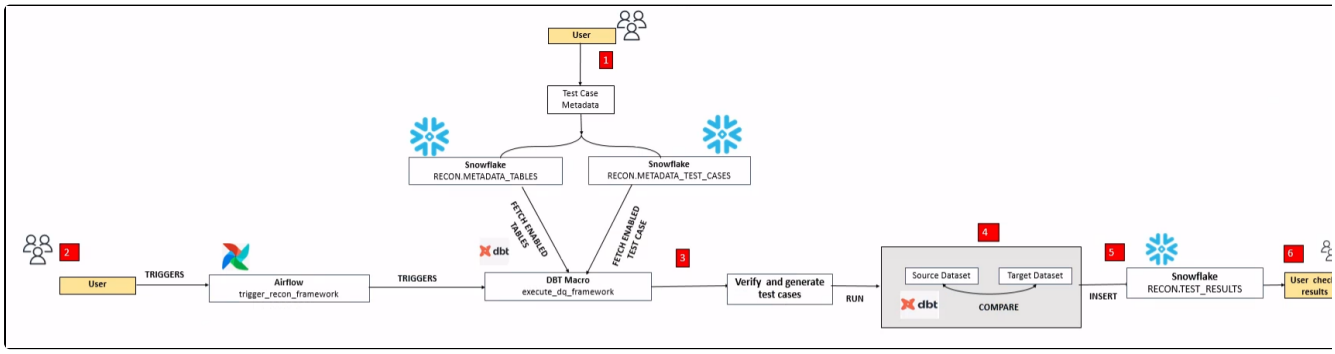
Note : Kafka and Five tran data reconciliation consider as out scope. This required for another design.

Source to landing data reconciliation work flow

Key Features

- **Built with Jinja-SQL Macros in dbt**
  Leverages dbt's templating capabilities for dynamic SQL generation and modular design.
- **Metadata-Driven Architecture**
  All test configurations and parameters are stored in centralized metadata tables, enabling easy access, validation, and modification across all data zones.
- **Configurable Test Case Execution**
  Supports toggle-based activation of individual test cases, allowing selective and fine-grained control over validation logic.
- **Automated Execution via Airflow**
  Integrated with Apache Airflow for scheduled runs, enabling continuous monitoring and anomaly detection across data pipelines.
- **Anomaly Diagnosis and Remediation**
  Post-execution insights help identify root causes of data discrepancies and facilitate targeted remediation.
- **Ease of Maintenance and Adoption**
  Designed for simplicity and modularity, making it straightforward to onboard, extend, and customize based on evolving business needs.

High level Architecture



Test Execution Workflow Overview

a. **Metadata Configuration**
   Users define and activate test scenarios by inserting metadata into the **RECON.METADATA_TABLES and RECON. METADATA_TEST_CASES** tables.
b. **Triggering the Framework**
   The user initiates the data quality process by triggering the Airflow DAG named **trigger_recon_framework**. This DAG invokes the parent dbt macro **execute_art_framework,** which orchestrates the test execution.
c. **Metadata Retrieval and Test Selection**
   The framework retrieves the enabled test scenarios and associated table configurations from the metadata tables.
d. **SQL Generation and Data Comparison**
   Based on the metadata, the framework dynamically generates SQL queries to extract source and target datasets. It then performs comparisons according to the defined validation logic.
e. **Result Logging**
   Upon completion of each test scenario, the framework logs the outcomes into the **RECON.TEST_RESULTS** table.
f. **Defect Analysis**
   Users can query the **TEST_RESULTS** table to review discrepancies, investigate anomalies, and initiate remediation steps as needed.

Key Technical Components

Database Design and Table Design

Data Reconciliation Framework database schema deployed in the below mentioned DB.

Platform Details	Zone Details
DB & Schema : LANDING__<ENV>.RECON **Environment Variables:** <ENV>: Environment identifier (DEV/SYST/PPTE/PROD)	**DB & Schema** : <ZONE>__LOGGING__<ENV>.RECON Example : CUST__LOGGING__DEV.RECON **Environment Variables:** <ZONE>: Deployment zone identifier(CUST,PTP,CUSTENG) <ENV>: Environment identifier (DEV/SYST/PPTE/PROD)

There are three key tables that are part of the Framework. Two of the tables are metadata tables (inputs) and one is a result table (output).

Within each Zone, this would sit under **RECON** Schema under the <zone>__logging__<env> Database.

These are further detailed below:

Table 1: Metadata_Tables

Purpose: Serves as the central metadata repository that stores source and target table configurations used in data reconciliation workflows

Table Definition

```
CREATE OR REPLACE TABLE <ZONE>__LOGGING__<ENV>.RECON.METADATA_TABLES (
    TABLE_ID NUMBER(38,0),
    LAYER VARCHAR(50),
    SOURCE_DATABASE_NAME VARCHAR(255),
    SOURCE_SCHEMA VARCHAR(255),
    SOURCE_TABLE VARCHAR(255),
    TARGET_DB_NAME VARCHAR(255),
    TARGET_SCHEMA VARCHAR(255),
    TARGET_TABLE VARCHAR(255),
    ENABLED VARCHAR(10),
    INSERTED_DATE TIMESTAMP_NTZ(9) DEFAULT CURRENT_TIMESTAMP(),
    UPDATED_DATE TIMESTAMP_NTZ(9)
);
```

Column Specifications

Column Name	Data Type	Nullable	Description
TABLE_ID	NUMBER(38,0)	No	Unique identifier for each table mapping
LAYER	VARCHAR(50)	Yes	Data layer classification (e.g., 'RAW', 'FOUNDATION', 'CURATED')
SOURCE_DATABASE_NAME	VARCHAR(255)	Yes	Name of the source database
SOURCE_SCHEMA	VARCHAR(255)	Yes	Source schema name
SOURCE_TABLE	VARCHAR(255)	Yes	Source table name
TARGET_DB_NAME	VARCHAR(255)	Yes	Name of the target database
TARGET_SCHEMA	VARCHAR(255)	Yes	Target schema name
TARGET_TABLE	VARCHAR(255)	Yes	Target table name
ENABLED	VARCHAR(10)	Yes	Status flag (Y/N)
INSERTED_DATE	TIMESTAMP_NTZ(9)	No	Record creation timestamp
UPDATED_DATE	TIMESTAMP_NTZ(9)	Yes	Record last modification timestamp

Business Rules

- TABLE_ID must be unique across the entire system
- ENABLED should contain only 'Y' or 'N' values
- Combination of (SOURCE_DATABASE_NAME, SOURCE_SCHEMA, SOURCE_TABLE) should be unique
- Combination of (TARGET_DB_NAME, TARGET_SCHEMA, TARGET_TABLE) should be unique

Table 2: Metadata_Test_Cases

Purpose: Acts as an advanced configuration repository for reconciliation test scenarios, offering support for custom SQL logic and enabling flexible, parameter-driven test definitions.

The final entries that are required into this table, depends on the detailed test case, and is detailed in sections further below in the document under section: Detailed Test Types

Table Definition

```
create or replace TABLE <ZONE>__LOGGING__<ENV>.RECON.METADATA_TEST_CASES (
    TEST_ID NUMBER(38,0),
    TABLE_ID NUMBER(38,0),
    TEST_NAME VARCHAR(16777216),
    TEST_TYPE VARCHAR(16777216),
    TEST_DESCRIPTION VARCHAR(16777216),
    ENABLED VARCHAR(16777216),
    SOURCE_SELECT_FIELDS VARCHAR(16777216),
    TARGET_SELECT_FIELDS VARCHAR(16777216),
    SOURCE_WHERE_CLAUSE VARCHAR(16777216),
    TARGET_WHERE_CLAUSE VARCHAR(16777216),
    AGGREGATE_FUNCTIONS VARCHAR(16777216),
    SOURCE_AGGREGATE_FIELDS VARCHAR(16777216),
    TARGET_AGGREGATE_FIELDS VARCHAR(16777216),
    SOURCE_UNIQUE_FIELDS VARCHAR(16777216),
    TARGET_UNIQUE_FIELDS VARCHAR(16777216),
    SOURCE_SQL VARCHAR(16777216),
    TARGET_SQL VARCHAR(16777216),
    ISCUSTOMSQL VARCHAR(16777216),
    DATA_DATE_VARIABLE VARCHAR(50),
    COMPARISON_TYPE VARCHAR(50),
    INCREMENTAL_COMPARISON_INTERVAL VARCHAR(50),
    SOURCE_DATE_FIELD VARCHAR(50),
    TARGET_DATE_FIELD VARCHAR(50),
    THRESHOLD_VALUE VARCHAR(16777216),
    INSERT_TIMESTAMP TIMESTAMP_NTZ(9),
    UPDATE_TIMESTAMP TIMESTAMP_NTZ(9)
);
```

Column Specifications

Column Name	Data Type	Nullable	Description
TEST_ID	NUMBER(38,0)	No	Unique identifier for each test case
TABLE_ID	NUMBER(38,0)	No	References parent table metadata
TEST_NAME	VARCHAR (16777216)	Yes	Human-readable name for the test case
TEST_TYPE	VARCHAR (16777216)	Yes	Type of reconciliation test being performed
TEST_DESCRIPTION	VARCHAR (16777216)	Yes	Detailed description of the test purpose and logic
ENABLED	VARCHAR (16777216)	Yes	Status flag indicating
SOURCE_WHERE_CLAUSE	VARCHAR (16777216)	Yes	WHERE clause conditions for source data filtering
TARGET_WHERE_CLAUSE	VARCHAR (16777216)	Yes	WHERE clause conditions for target data filtering
AGGREGATE_FUNCTIONS	VARCHAR (16777216)	Yes	Aggregation functions to apply (SUM, COUNT, AVG, MIN, MAX)
SOURCE_SELECT_FIELDS	VARCHAR (16777216)	Yes	Fields to select in source query
TARGET_SELECT_FIELDS	VARCHAR (16777216)	Yes	Fields to select in target query
SOURCE_AGGREGATE_FIELDS	VARCHAR (16777216)	Yes	Fields to aggregate in source query
TARGET_AGGREGATE_FIELDS	VARCHAR (16777216)	Yes	Fields to aggregate in target query
SOURCE_UNIQUE_FIELD	VARCHAR (16777216)	Yes	source Fields that should be unique for duplicate detection
TARGET_UNIQUE_FIELD	VARCHAR (16777216)	Yes	Target Fields that should be unique for duplicate detection
SOURCE_SQL	VARCHAR (16777216)	Yes	Complete custom SQL query for source data
TARGET_SQL	VARCHAR (16777216)	Yes	Complete custom SQL query for target data
ISCUSTOMSQL	VARCHAR (16777216)	Yes	Flag indicating if custom SQL is used ('Y'/'N')
DATA_DATE_VARIABLE	VARCHAR(50)	Yes	dbt variable name for data date
COMPARISON_TYPE	VARCHAR(50)	Yes	full, incremental
INCREMENTAL_COMPARISON_INTERVAL	VARCHAR(50)	Yes	daily, weekly, monthly

SOURCE_DATE_FIELD	VARCHAR(50)	Yes	used to specify which date columns should be used for incremental date filtering used to specify which date columns should be used for freshness
TARGET_DATE_FIELD	VARCHAR(50)	Yes	used to specify which date columns should be used for incremental date filtering used to specify which date columns should be used for freshness
THRESHOLD_VALUE	VARCHAR(50)	Yes	Variance value between source and target row counts before a test is considered a failure
INSERT_TIMESTAMP	TIMESTAMP_NTZ (9)	Yes	Record creation timestamp
UPDATE_TIMESTAMP	TIMESTAMP_NTZ (9)	Yes	Record last modification timestamp

Enhanced Test Types and Configurations

Standard Configuration Tests

Test	Configuration Mode	Key Fields(METADATA_TEST_CASES)
Row Count Check	Standard	SOURCE_SELECT_FIELDS, TARGET_SELECT_FIELDS, WHERE_CLAUSES
Aggregation Check	Standard	AGGREGATE_FUNCTIONS, SOURCE_AGGREGATE_FIELDS, TARGET_AGGREGATE_FIELDS
Freshness Check	Standard	SOURCE_DATE_FIELD, TARGET_DATE_FIELD
Unique Active Check	Standard	SOURCE_UNIQUE_FIELD,TARGET_UNIQUE_FIELD
Source File Row Count Check	Standard	SOURCE_SELECT_FIELDS,TARGET_SELECT_FIELDS

Custom Configuration Tests

Test	Configuration Mode	Key Fields(METADATA_TEST_CASES)
Row Count Check	Custom SQL	SOURCE_SQL, TARGET_SQL, ISCUSTOMSQL='Y'

Business Rules and Validation

- TEST_ID must be unique across the entire system
- TABLE_ID must reference a valid table metadata entry
- ENABLED should contain only 'Y' or 'N' values
- ISCUSTOMSQL should contain only 'Y' or 'N' values

Configuration Logic Rules

- Rule 1: If ISCUSTOMSQL = 'Y', then SOURCE_SQL and TARGET_SQL must be populated
- Rule 2: If ISCUSTOMSQL = 'Y', then standard configuration fields should be used
- Rule 3: Custom SQL takes precedence over standard configuration when both exist

Test Type Validation

- Standard Tests: Require appropriate configuration fields based on test type
- Custom SQL Tests: Require SOURCE_SQL, TARGET_SQL, and ISCUSTOMSQL='Y'

Table 3: Test_Results

Purpose: Provides an enhanced results repository with extended tracking functionality for process execution dates, enabling more effective monitoring and auditability of reconciliation activities.

Table Definition

```
create or replace TABLE <ZONE>__LOGGING__<ENV>.RECON.TEST_RESULTS (
    TEST_EXECUTION_ID VARCHAR(50),
    TEST_ID NUMBER(38,0),
    TABLE_ID NUMBER(38,0),
    SOURCE_TABLE_NAME VARCHAR(16777216),
    TARGET_TABLE_NAME VARCHAR(16777216),
    TEST_TYPE VARCHAR(16777216),
    SOURCE_RESULT VARCHAR(16777216),
    TARGET_RESULT VARCHAR(16777216),
    DESCRIPTION VARCHAR(16777216),
    TEST_PASSED BOOLEAN,
    THRESHOLD_STATUS VARCHAR(50),
    VALIDATION_MSG VARCHAR(16777216),
    VALUE_DIFFERENCE NUMBER(10,0),
    VARIANCE_PERCENTAGE NUMBER(10,0),
    RESULT_QUERY VARCHAR(16777216),
    LAST_PROCESS_DATE DATE,
    INSERT_TIMESTAMP TIMESTAMP_LTZ(9)
);
```

Column Specifications

Column Name	Data Type	Nullable	Description
TEST_EXECUTION_ID	VARCHAR(50)	No	Unique identifier for each test execution
TEST_ID	NUMBER(38,0)	No	References METADATA_TEST_CASES.TEST_ID
TABLE_ID	VARCHAR(16777216)	Yes	References table METADATA_TABLES.TABLE_ID
SOURCE_TABLE_NAME	VARCHAR(16777216)	Yes	Full name or description of source data
TARGET_TABLE_NAME	VARCHAR(16777216)	Yes	Full name or description of target data
TEST_TYPE	VARCHAR(16777216)	Yes	Type of test executed (copied from test case)
SOURCE_RESULT	VARCHAR(16777216)	Yes	Result value from source query execution
TARGET_RESULT	VARCHAR(16777216)	Yes	Result value from target query execution
DESCRIPTION	VARCHAR(16777216)	Yes	Execution description or additional context
TEST_PASSED	BOOLEAN	Yes	**TRUE** if test passed, **FALSE** if failed, **FALSE** for errors
VALIDATION_MSG	VARCHAR(16777216)	Yes	Detailed Validation message
THRESHOLD_STATUS	VARCHAR(16777216)	Yes	Provides more granular information
VALUE_DIFFERENCE	VARCHAR(16777216)	Yes	Absolute difference between source and target results
VARIANCE_PERCENTAGE	NUMBER(10,0)	Yes	Percentage variance between source and target
RESULT_QUERY	VARCHAR(16777216)	Yes	Complete SQL Query used for validation
INSERT_TIMESTAMP	TIMESTAMP_LTZ(9)	Yes	Test execution completion timestamp
LAST_FILE_PROCESSED_DATE	DATE	Yes	Last file Processed date used for source file to landing data comparison
DATA_DATE	DATE	Yes	dbt parameter value is captured in this field

The **last_file_process_date** field tracks the last successful reconciliation date based on the watermark column. It helps determine which source data extraction dates still need to be processed into the landing layer.
During each execution, the **data_date** parameter value is captured in Data_date field and used as the reference point used for reporting this mechanism ensures seamless data processing across all layers in the pipeline:
Landing Raw Foundation Curate

Data Rules

- TEST_EXECUTION_ID format: 'ART__YYYYMMDD_HHMMSS'
- TEST_ID must exist in METADATA_TEST_CASES

Status Message

S. NO	Comparison	Validation Check	Validation Message	Thershold Status	Example
1	Table-to-Table Comparison	Row Count	• ROW_COUNT_MATCHED • ROW_COUNT_MISMATCHED	• WITHIN_THRESHOLD • EXCEEDS_THRESHOLD	Count(col)

2	Table-to-Table Comparison	Aggregation	• AGGREGATION_MATCHED • AGGREGATION_MISMATCHED	• WITHIN_THRESHOLD • EXCEEDS_THRESHOLD	Sum(col),Min(col),Max(col), Avg(col)
3	Table-to-Table Comparison	Freshness	• SOURCE_DATA_STALE • TARGET_DATA_STALE • DATA_FRESH_ROW_COUNT_MISMATCHED • DATA_FRESH_ROW_COUNT_MATCHED	• WITHIN_THRESHOLD • EXCEEDS_THRESHOLD • NO_DATA	Max(DateCol) ,Count (DateCol)
4	Table-to-Table Comparison	Uniqueness	• UNIQUE_COUNT_MATCHED • UNIQUE_COUNT_MISMATCHED • NO_DATA	• WITHIN_THRESHOLD • EXCEEDS_THRESHOLD • UNIQUE_COUNT_MATCHED	Count(Distinct(col))
5	Source Manifest to Landing	Source File	• ROW_COUNT_MATCHED • ROW_COUNT_MISMATCHED	• WITHIN_THRESHOLD • EXCEEDS_THRESHOLD	Count(col),Sum(col)

DBT Project flow

Macro Creation & Development

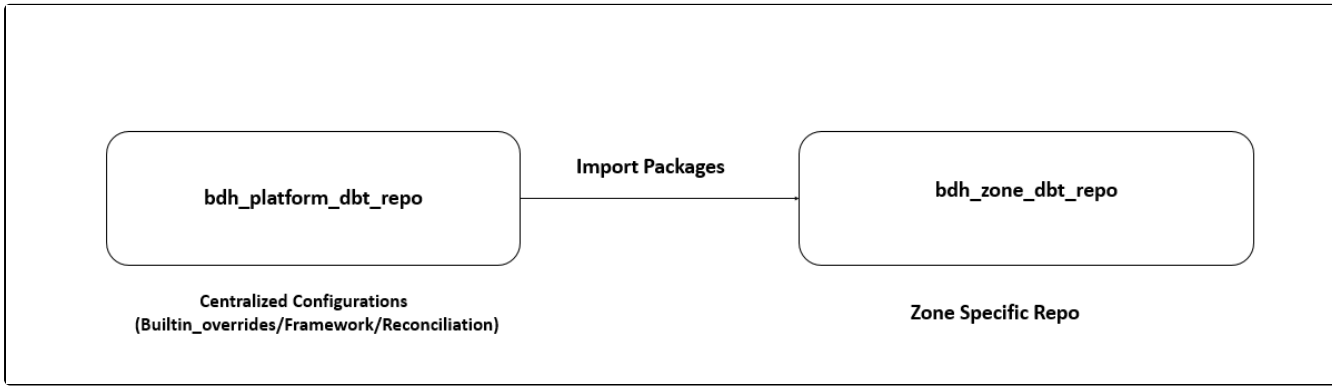
Reusable dbt macros are created and managed within the **bdh_platform_dbt_repo** project, where they are independently version-controlled and tested in a centralized repository.

DBT Project Setup: Reusable Macros, Model Naming, and Execution Flow - Digital, Data & Analytics - BNZ Confluence

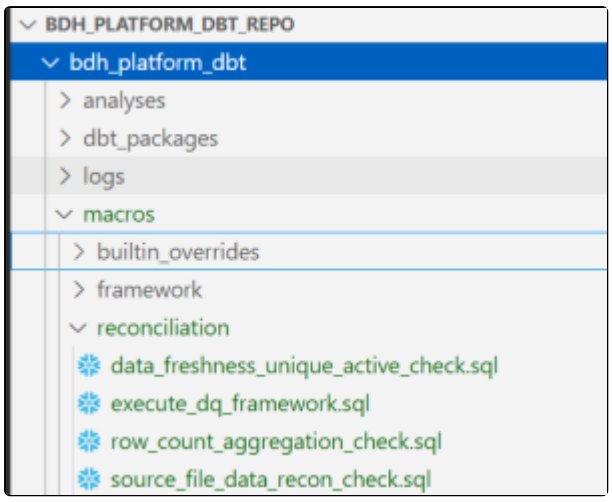
Distribution & Import

The recon macros in **bdh_platform_dbt_repo** are packaged and published for reuse across dbt projects. Target projects import these macros via dbt's package system, enabling them to reference shared logic directly from the central library—eliminating the need for code duplication.

Projects integrate the shared macro library by specifying it in the **packages.yml** configuration file. Running **dbt deps** installs the required dependencies, making the macros accessible within the project. Macro behavior can be tailored through parameterization, allowing customization without altering the core logic.

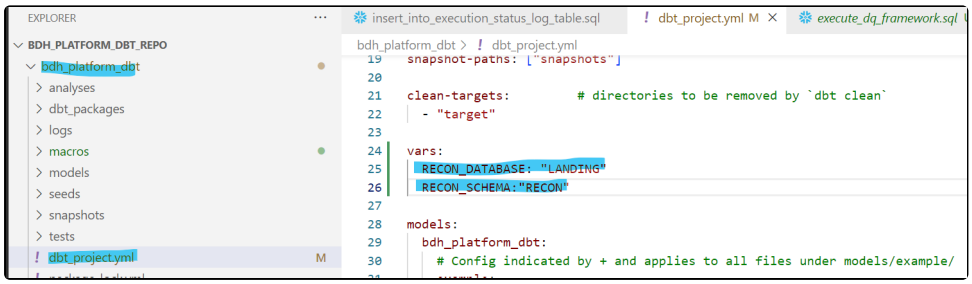


Platform(bdh_platform_dbt_repo)	Zone A (



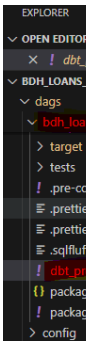
Configured variables

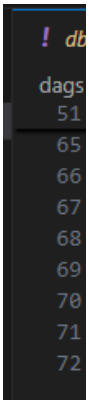
dbtproject.yml



Configur

dbtprojec





As part of the **Reconciliation below** metadata and results tables are created in Snowflake DB(Zone, Platform) using **dbt seed files**.

The following tables are provisioned automatically via `dbt seed` execution

- `RECON.METADATA_TEST_CASES`
- `RECON.METADATA_TABLES`
- `RECON.TEST_RESULTS`

How it works

1. **Seed CSV Files** are created under the seed/ directory in the dbt project:
   - <zone>___recon___metadata_test_cases.csv
   - <zone>___recon___metadata_tables.csv
   - <zone>___recon___test_results.csv
2. **Headers in the CSV files** define the table column names.
3. **Column data types** and configurations are maintained in the corresponding **schema.yml** file (e.g., **seeds.yml**).
4. When the command `dbt seed` is executed:
   - dbt reads the CSV files.
   - Creates the target tables in Snowflake DB under the schema **RECON**.
   - Loads the data from the CSV into these tables.

Core Macro Framework Objects

1.Execute_art_framework

The main orchestration macro that coordinates all data quality testing activities.

Purpose:

Central macro responsible for managing and executing all data quality test cases.

- Retrieves all active test cases from metadata
- Dynamically constructs and runs SQL for each test
- Logs results into a designated results table

Parameters:

- **Execution_id** (optional): Unique identifier for the test execution. Automatically generated if not supplied.

Functionality:

- Creates a unique execution ID using the current timestamp
- Loads configuration details from metadata tables
- Iterates through each enabled test case
- Invokes the corresponding child macro based on test type
- Records results in the **TEST_RESULTS** table

Logic:

- Query metadata from:
  - **RECON.METADATA_TABLES** for table definitions
  - **RECON.METADATA_TEST_CASES** for test configurations
- For each test case:
  - Identify the test type via **test_name**
  - Call the appropriate macro to execute the test
- Supported test types include:
  - Row Count Check
  - Aggregation Check
  - Freshness Check
  - Unique Active Check
- Store results in **RECON.TEST_RESULTS**, capturing:
  - Execution ID
  - Test name
  - Executed SQL query
  - Source and target record counts
  - Timestamp
  - Test status (Pass/Fail)

2.Row_count_aggregation_check

Purpose:

Validates data consistency between source and target tables by comparing row counts and aggregated values. Supports both row-level validation and aggregation-level reconciliation using group-by and aggregate fields. Ensures integrity of data post-transformation or load.

Parameters:

- **test_name:** Human-readable name for the test case
- **test_type**: Type of validation to perform **(row_count_check, aggregation_check)**
- **source_table**: Fully qualified name of the source table
- **target_table**: Fully qualified name of the target table
- **source_where_clause (optional)**: Filter conditions for source data
- **target_where_clause (optional)**: Filter conditions for target data
- **source_select_fields (optional)**: Grouping columns for source aggregation
- **target_select_fields (optional)**: Grouping columns for target aggregation
- **source_aggregate_fields (optional)**: Columns to aggregate in source
- **target_aggregate_fields (optional)**: Columns to aggregate in target
- **aggregate_functions (optional)**: Aggregation functions to apply (sum, count, avg, min, max)
- **source_sql :** Complete custom SQL query for source data
- **target_sql :** Complete custom SQL query for Target data
- **iscustomsql :** Flag indicating if custom SQL is used ('Y'/'N')
- **threshold_value (optional)**: Acceptable variance threshold (default: 1%)
- **data_date_variable :** dbt variable name for data date
- **comparison_type :** full, incremental
- **incremental_comparison_interval :** daily, weekly, monthly
- **source_date_field :** used to specify which date columns should be used for incremental date filtering
- **target_date_field:** used to specify which date columns should be used for incremental date filtering

Key Features:

- **Dynamic Column Parsing**: Automatically trims and parses comma-separated column lists
- **Flexible Aggregation**: Supports multiple aggregation functions for versatile validation

- **Tolerance Management**: Allows configurable variance thresholds to align with business rules
- **Comprehensive Comparison**: Uses **EXCEPT** logic to detect mismatches between source and target
- **Detailed Metrics**: Outputs variance percentages, differences, and validation status

Logic:

- **threshold_value (optional)**: Acceptable variance threshold (default: 1%).
- Apply filtering conditions to both source and target datasets.
- Perform group-by and aggregation operations as defined.
- Compare aggregated results between source and target.
- Return metrics including:
    - Row count
    - Value differences
    - Validation status (pass/fail)
    - Percentage variance

3.Data_freshness_unique_active_check

Purpose:

Validates data quality by performing two key checks:

- **Freshness Check** to ensure data is updated within acceptable timeframes (SLA compliance)
- **Uniqueness Check** to verify record integrity and enforce uniqueness constraints

Parameters:

- **test_name:** Human-readable name for the test case
- **test_type**: Type of validation (Freshness Check, Active Check)
- **source_table**: Fully qualified name of the source table
- **target_table**: Fully qualified name of the target table
- **source_where_clause (optional)**: Filter conditions for source data
- **target_where_clause (optional)**: Filter conditions for target data
- **source_freshness_column** : used to specify which date columns should be used for freshness
- **target_freshness_column:** used to specify which date columns should be used for freshness
- **threshold_value** : Maximum allowed data age in hours (example 24)
- **source_unique_fields :** used to specify Column for source field
- **target_unique_fields :** used to specify Column for Target field
- **source_sql** : Complete custom SQL query for source data
- **target_sql :** Complete custom SQL query for Target data
- **data_date_variable :** dbt variable name for data date

Freshness Check Features:

- **Time-Based Validation**: Compares latest timestamp against current time
- **Stale Data Detection**: Flags records older than the defined threshold
- **Fresh Record Counting**: Counts records within the freshness window
- **Temporal Metrics**: Calculates hours since last update
- **Cross-System Comparison**: Ensures timestamp consistency between source and target

Active/Uniqueness Check Features:

- **Multi-Column Validation**: Supports composite uniqueness across multiple fields
- **Duplicate Detection**: Identifies and counts duplicate records
- **Null Value Analysis**: Tracks nulls in uniqueness columns
- **Combination Validation**: Ensures uniqueness of column combinations
- **Detailed Breakdowns**: Provides per-column uniqueness statistics

Logic:

- **Freshness Check:**
    - Retrieve `MAX(Source_freshness_column)` from source
    - Compare with current timestamp
    - If the difference exceeds `freshness_threshold_hours`, mark as failed
- **Uniqueness Check:**
    - Perform `GROUP BY` on `unique_columns`
    - Identify duplicate groups and null values
    - Flag violations based on uniqueness rules

4.Source_file_data_recon_check

Purpose:

Executes end-to-end data reconciliation between source metadata and target tables to ensure integrity throughout ETL workflows.
Validates data transfer success by comparing record counts and key column checksums from source file metadata against actual target table contents.
Supports both full-load and incremental reconciliation scenarios with detailed variance analysis and status reporting.

Parameters

- **test_name**: Unique identifier for the reconciliation test run

- **test_type**: Type or classification of the reconciliation check
- **test_id** : References METADATA_TEST_CASES.TEST_ID
- **source_table**: Metadata table containing source file processing details
- **target_table**: Target data table to be validated against source metadata
- **source_where_clause**: Optional filter conditions for source metadata
- **target_where_clause**: Optional filter conditions for target data
- **target_select_fields :** Grouping columns for target aggregation
- **threshold_value :** Acceptable variance threshold
- **source_date_field :** used to specify which date columns should be used for incremental date filtering
- **target_date_field :** used to specify which date columns should be used for incremental date filtering
- **data_date_variable :** dbt variable name for data date

Key Features

- **Metadata-Driven Validation:** Compares precomputed source metrics (record counts, checksums) with actual target table statistics
- **Dual Validation Strategy**: Ensures both row count accuracy and data integrity via checksum comparisons
- **Incremental Support**: Filters data by latest extraction date for daily reconciliation when incremental mode is enabled
- **Dynamic Key Handling**: Parses and trims comma-separated key column lists for flexible configuration
- **Detailed Reporting**: Outputs pass/fail status, variance percentages, and descriptive error messages
- **Configurable Date Filtering**: Uses **extraction_date** (source) and **load_date** (target) for temporal alignment

Logic

- Apply optional filters to both source metadata and target data tables.
- Extract source metrics (record count, key column checksum) for the specified date range.
- Compute target metrics (record count, checksum) from actual data.
- Compare source and target metrics, calculating variances.
- Generate a reconciliation report with:
  - Validation status
  - Record count differences
  - Checksum mismatches
  - Variance percentages
  - Descriptive messages for diagnostics

Orchestration

Purpose: With an Airflow DAG to orchestrate data quality checks and reconciliation processes..

DAG Name: dbt_excute_art_framework

Primary Function:
Invokes the core macro **execute_art_framework()** to initiate the data quality framework.

Role & Capabilities:
Serves as the orchestration layer responsible for:

- Scheduling and triggering data quality checks
- Managing task execution and workflow coordination
- Handling task dependencies and sequencing
- Enabling monitoring and alerting for operational visibility

DAG File Location: /usr/local/airflow/dags/recon_dag.py

Execution Modes:

- **Scheduled** (via Airflow scheduler)
- **Manual** (on-demand trigger)

Reconciliation RBAC

Purpose

RBAC (Role-Based Access Control) ensures secure, efficient, and structured access to data and metadata by restricting user permissions based on roles (e.g., Data Steward, Data Engineer, Business Analyst), aligning with organizational responsibilities. Users are only exposed to functionalities and data relevant to their role, simplifying their interactions with the tool.

It enhances data security by preventing unauthorized access. RBAC plays a critical role in compliance with data protection regulations, facilitating accountability by clearly defining user actions. It simplifies access management, by assigning permissions to roles rather than individuals, enabling scalability and reducing administrative overhead.

Create AR roles for different access levels

Platform	Zone

BDH_SF_SVC_AR_<PLATFRM>_<ENV>_WRITE	BDH_SF_SVC_AR_<ZONE>_<ENV>_WRITE
BDH_SF_SVC_AR_<PLATFRM>_<ENV>_READ	BDH_SF_SVC_AR_<ZONE>_<ENV>_READ
BDH_SF_SVC_AR_<PLATFRM>_<ENV>_MANAGE	BDH_SF_SVC_AR_<ZONE>_<ENV>_MANAGE
BDH_SF_SV_AR_<PLATFRM>_<ENV>_CREATE_OBJECTS	BDH_SF_SV_AR_<ZONE>_<ENV>_CREATE_OBJECTS

Grant permissions to **RECON** Schema

Privileges	Permissions
WRITE	database: usage schema: usage table: insert, update, delete
MANAGE	database: usage schema: create table, create view, modify, usage
READ	database: usage schema: usage table: select view: select
CREATE_OBJECTS	database: usage schema: create table, create view, usage

Create Functional Roles

Platform	Zone
BDH_SF_FR_LANDING_<ENV>_DEVELOPER BDH_SF_FR_LANDING_<ENV>_SUPPORT_GEN BDH_SF_FR_LANDING_<ENV>_SUPPORTADMIN BDH_SF_SVC_FR_PLATFRM_<ENV>_AIRFLOW BDH_SF_SVC_FR_PLATFRM_<ENV>_FLYWY	BDH_SF_FR_<ZONE>_<ENV>_DEVELOPER BDH_SF_FR_<ZONE>_<ENV>_SUPPORT_GEN BDH_SF_FR_<ZONE>_<ENV>_SUPPORTADMIN BDH_SF_SVC_FR_<ZONE>_<ENV>_AIRFLOW BDH_SF_SVC_FR_<ZONE>_<ENV>_FLYWY

Create AR TO FR Mapping

Note : Via zone automation AR to FR role mapping will be assigned as per the existing setup .

Create FR TO FG Mapping

Note : Via zone automation FR to FG role mapping will be assigned as per the existing setup .

Data Reconciliation: Test Types, Metadata Setup, and Execution Result

Detailed Test Types and Functionality

Below listed are the main groupings of test types. Each of this has multiple variations, and this is detailed within each of the parent test type.

Below link provide the details unit testing results.

DBT Recon Framework - Unit Testing Doc - Digital, Data & Analytics - BNZ Confluence

S. NO	Test Type	Test Subtype	Applicable Layers	Macro Name	Objective	Implementation	Success Criteria

1	Row Count Validation	• Basic row count check • Row count with WHERE filter • Row count with GROUP BY • Row count with filter + GROUP BY • (Daily/Weekly/Monthly) incremental row count • (Daily/Weekly/Monthly) incremental with WHERE filter • (Daily/Weekly/Monthly) incremental with GROUP BY • (Daily/Weekly/Monthly) incremental with filter + GROUP BY	Landing Raw Foundation Curated	row_count_aggregation_check	Ensure source and target tables contain matching row counts	• Compares total row counts between source and target • Supports optional WHERE clause filters and GROUP BY logic	Row counts match within an acceptable variance threshold
2	Aggregation Validation	• Basic aggregation check • Aggregation with WHERE filter • Aggregation with GROUP BY • Aggregation with filter + GROUP BY • (Daily/Weekly/Monthly) incremental aggregation • (Daily/Weekly/Monthly) incremental aggregation with WHERE filter • (Daily/Weekly/Monthly) incremental aggregation with GROUP BY • (Daily/Weekly/Monthly) incremental aggregation with filter + GROUP BY	Landing Raw Foundation Curated	row_count_aggregation_check	Verify consistency of aggregated metrics across source and target datasets	• Performs comparisons using aggregate functions such as SUM, COUNT, AVG, etc.	Aggregated values align within predefined tolerance levels
3	Data Freshness Validation	• Check data freshness with data variable parameter • Check data freshness with filter condition • Check data freshness without data variable parameter • Check data freshness without filter condition	Landing Raw Foundation Curated	data_freshness_unique_active_check	Confirm that data is current and falls within expected update windows	• Evaluates timestamp fields against configured freshness thresholds (e.g., hours)	Most recent timestamp meets or exceeds freshness requirements

4	Unique Active Record Validation	• Check data uniqueness with data variable parameter • Check data uniqueness with filter condition • Check data uniqueness without data variable parameter • Check data uniqueness without filter condition	Landing Raw Foundation Curated	data_freshness_unique_active_check	Enforce uniqueness constraints and validate active record integrity	• Checks for duplicate entries in designated unique fields • Validates counts of active records based on business logic	No duplicates present in fields expected to be unique
5	Source File Data Reconciliation Check	• Basic row count check • Row count with WHERE filter	Source Landing	source_file_row_count_check	Validates data integrity between source and target tables by comparing record counts and key checksums to ensure successful data transfers	• Compares record counts between source extraction files and target loaded data • Generates key checksums using specified key columns to validate data completeness • Supports both full load and incremental daily load reconciliation patterns	Source record count matches target record count exactly and checksum values are identical between source and target then overall status returns 'PASS' for both validations

Metadata Tables Configuration

Mandatory fields for specific sub-type of a test case with sample values.

Each row represents a specific test case and shows whether a value needs to be entered in seed file for the test case excution.

TEST TYPE	TEST SUB TYPE	TEST CASE DESCRIPTION	ENABLED	TEST_ID	TABLE_ID	TEST_NAME	SOURCE_SELECT_FIELDS	TARGET_SELECT_FIELDS	SOURCE_WHERE_CLAUSE	TARGET_WHERE_CLAUSE	AGGREGATE_FUNCTIONS	SOURCE_AGGREGATE_FIELDS	TARGET_AGGREGATE_FIELDS	SOURCE_UNIQUE_FIELDS	TARGET_UNIQUE_FIELDS	SOURCE_SQL	TARGET_SQL	ISCUSTOMSQL	DATA_DATE_VARIABLE	COMPARISON_TYPE	INCREMENTAL_COMPARISON_INTERVAL	SOURCE_DATE_FIELD	TARGET_DATE_FIELD	THRESHOLD_VALUE
Row Count Validation																		N	RUN_DATE					
	Basic row count check	Compare row counts between source and target	Y	101	1	Row Count Check												N	RUN_DATE	full				1
	Row count with WHERE filter	Compare row counts with filtering conditions	Y	102	8	Row Count Check			status = 'ACTIVE'	is_active = 1								N	RUN_DATE	full				2
	Row count with GROUP BY	Compare grouped row counts	Y	103	4	Row Count Check	store_id, product_category, transaction_date	store_id, product_category, transaction_date			sum	quantity	quantity					N	RUN_DATE	full				0.5
	Row count with filter + GROUP BY	Compare grouped row counts with filtering	Y	104	10	Row Count Check	category	product_category	is_deleted = 0	deleted_flag = 'N'		category	product_category					N	RUN_DATE	full				5
																			RUN_DATE					
	(Daily /Weekly /Monthly) incremental row count	incremental row count check	Y	105	9	Row Count Check												N	RUN_DATE	incremental	daily	order_date	processed_date	2

	(Daily /Weekly /Monthly) incremental with WHERE filter	incremental with filtering conditions	Y	106	9	Row Count Check	Y	Y	order_date >= ${DATA_DATE}	processed_date >= ${DATA_DATE}								N	RUN_DATE	incremental	daily	order_date	order_date	0.5
	(Daily /Weekly /Monthly) incremental with GROUP BY	incremental with grouping	Y	107	Y	Row Count Check	store_id, product_category, transaction_date	store_id, product_category, transaction_date			sum	quantity	quantity					N	RUN_DATE	incremental	daily	source_dte_col	target_date_col	1
	(Daily /Weekly /Monthly) incrementalwith filter + GROUP BY	incremental with filtering and grouping	Y	108	10	Row Count Check	category	product_category	is_deleted = 0	deleted_flag = 'N'	count	category	product_category					N	RUN_DATE	incremental	daily	order_date	processed_date	5
	Basic Custom SQL row count check	Compare row counts between source and target	Y	109	8	Row Count Check										SELECT customer_id, customer_name FROM LANDING__DEV.IBIS.CUSTOMER_DATA WHERE status = 'ACTIVE'	SELECT cust_id, customer_name FROM CUST__RAW__DEV.IBIS.CUSTOMER_PROCESSED WHERE is_active = 1	Y	RUN_DATE					0.5
	Basic Row count with WHERE filter	Compare row counts with filtering conditions	Y	110	2	Row Count Check			status = 'ACTIVE'	is_active = 1						SELECT s.product_id, s.quantity, s.revenue, p.product_name FROM LANDING__DEV.IBIS.sales_source s JOIN LANDING__DEV.IBIS.products_source p ON s.product_id = p.id JOIN LANDING__DEV.IBIS.regions_source r ON s.region_id = r.id WHERE region_name = 'North'	SELECT product_id, quantity, revenue, product_name FROM CUST__RAW__DEV.IBIS.sales_target	Y	RUN_DATE					1
																		N	RUN_DATE					
Aggregation Validation	Basic aggregation check	Compare aggregate values between source and target	Y	112	4	Aggregation Check	product_category	product_category			sum	quantity	quantity					N	RUN_DATE	full				1
	Aggregation with WHERE filter	Compare aggregate values with filtering conditions	Y	113	4	Aggregation Check	product_category	product_category	PRODUCT_CATEGORY='Electronics'	PRODUCT_CATEGORY='Electronics'	count	quantity, sales_amount	quantity, sales_amount					N	RUN_DATE	full				2
	Aggregation with GROUP BY	Compare grouped aggregate values	Y	114	4	Aggregation Check	product_category	product_category			avg	sales_amount	sales_amount					N	RUN_DATE	full				0.5
	Aggregation with filter + GROUP BY	Compare grouped aggregate values with filtering	Y	115	4	Aggregation Check	product_category	product_category	PRODUCT_CATEGORY='Electronics'	PRODUCT_CATEGORY='Electronics'	max	sales_amount	sales_amount					N	RUN_DATE	full				5
																		N	RUN_DATE					
	(Daily /Weekly /Monthly) incremental aggregation	incremental aggregate check	Y	116	4	Aggregation Check	product_category	product_category			sum	quantity	quantity					N	RUN_DATE	incremental	Daily /Weekly /Monthly	datecolumn	datecolumn	1
	(Daily /Weekly /Monthly) incremental aggregation with WHERE filter	incremental aggregate with filtering conditions	Y	117	4	Aggregation Check	product_category	product_category	PRODUCT_CATEGORY='Electronics'	PRODUCT_CATEGORY='Electronics'	count	quantity, sales_amount	quantity, sales_amount					N	RUN_DATE	incremental	Daily /Weekly /Monthly	datecolumn	datecolumn	2
	(Daily /Weekly /Monthly) incremental aggregation with GROUP BY	incremental aggregate with grouping	Y	118	4	Aggregation Check	product_category	product_category			avg	sales_amount	sales_amount					N	RUN_DATE	incremental	Daily /Weekly /Monthly	datecolumn	datecolumn	0.5

	(Daily /Weekly /Monthly) incremental aggregation with filter + GROUP BY	incremental aggregate with filtering and grouping	Y	119	4	Aggregation Check	product_category	product_category	PRODUCT_CATEGORY='Electronics'	PRODUCT_CATEGORY='Electronics'	max	sales_amount	sales_amount					N	RUN_DATE		incremental	Daily /Weekly /Monthly	datecolumn	datecolumn	5
																		N	RUN_DATE						
Data Freshness Validation	Check data freshness with data variable parameter	Data_Freshness_Check	Y	120	2	Freshness Check			CREATED_TIMESTAMP >= ${DATA_DATE}	PROCESSED_TIMESTAMP >= ${DATA_DATE}								N	RUN_DATE				UPDATED_TIMESTAMP	LAST_UPDATED	24
	Check data freshness with filter condition	Data_Freshness_Check	Y	121	6	Freshness Check			status = 'ACTIVE'	is_active = '1'								N	RUN_DATE				UPDATED_TIMESTAMP	LAST_UPDATED	12
	Check data freshness without data variable parameter	Data_Freshness_Check	Y	123	7	Freshness Check												N	RUN_DATE				UPDATED_TIMESTAMP	LAST_UPDATED	1
	Check data freshness without filter condition	Data_Freshness_Check	Y	124	8	Freshness Check												N	RUN_DATE				UPDATED_TIMESTAMP	LAST_UPDATED	2
																		N	RUN_DATE						
Unique Active Record Validation	Check data uniqueness with data variable parameter	Data_uniqueness_Check	Y	125	11	Unique Active Check			status = 'ACTIVE' AND CREATED_TIMESTAMP >= ${DATA_DATE}	is_active = '1' AND PROCESSED_TIMESTAMP >= ${DATA_DATE}								N	RUN_DATE						Y
	Check data uniqueness with filter condition	Data_uniqueness_Check	Y	126	12	Unique Active Check			order_status IN ('COMPLETED', 'PENDING', 'PROCESSING')	status IN ('PROCESSED', 'PROCESSING')				ID	ID			N	RUN_DATE						1
	Check data uniqueness without data variable parameter	Data_uniqueness_Check	Y	127	11	Unique Active Check								store_id	store_id			N	RUN_DATE						2
	Check data uniqueness without filter condition	Data_uniqueness_Check	Y	128	12	Unique Active Check								ID	ID			N	RUN_DATE						0.1
														store_id	store_id			N	RUN_DATE						0.8
Source File Validation	Basic row count check	Compare row counts between source and target	Y	129	13	Source file Row count check			ID, region_name	table_name = 'regions_source'								N	RUN_DATE						
	Row count with WHERE filter	Compare row counts with filtering conditions	Y	130	14	Source file Row count check			ID, region_name	table_name = 'regions_source'								N	RUN_DATE						

Mandatory Columns – `METADATA_TEST_CASES Table`

1. **ISCUSTOMSQL**
   - Values: `'Y'` (custom SQL), `'N'` (standard).
   - Purpose: Identifies if test case uses custom SQL.
2. **DATA_DATE_VARIABLE**
   - Value: `'RUN_DATE'` (fixed).
   - Purpose: Standard placeholder for processing date.

3. **When `COMPARISON_TYPE = 'INCREMENTAL' and INCREMENTAL_COMPARISON_INTERVAL='Daily' or 'Weekly' or 'Monthly'`**
    - **Mandatory Columns**: SOURCE_DATE_FIELD, TARGET_DATE_FIELD
4. **When `TEST_NAME = 'Freshness Check'`**

    - **Mandatory Columns**: SOURCE_DATE_FIELD, TARGET_DATE_FIELD

`Sample screen shot - Mandatory Data fields`

	ISCUSTOMSQL	DATA_DATE_VARIABLE	COMPARISON_TYPE	INCREMENTAL_COMPARISON_INTERVAL	SOURCE_DATE_FIELD	TARGET_DATE_FIELD
1	N	run_date	incremental	daily	LOAD_DATE	DWH_EFFECTIVE_FROM_TSTAM
2	N	run_date	incremental	weekly	LOAD_DATE	DWH_EFFECTIVE_FROM_TSTAM
3	N	run_date	incremental	monthly	LOAD_DATE	DWH_EFFECTIVE_FROM_TSTAM
4	Y	run_date	incremental	weekly	LOAD_DATE	DWH_EFFECTIVE_FROM_TSTAM

Sample Screenshots including Configuration & Test Execution Results

The above configuration when implemented against the metadata tables are seen as screenshots below.

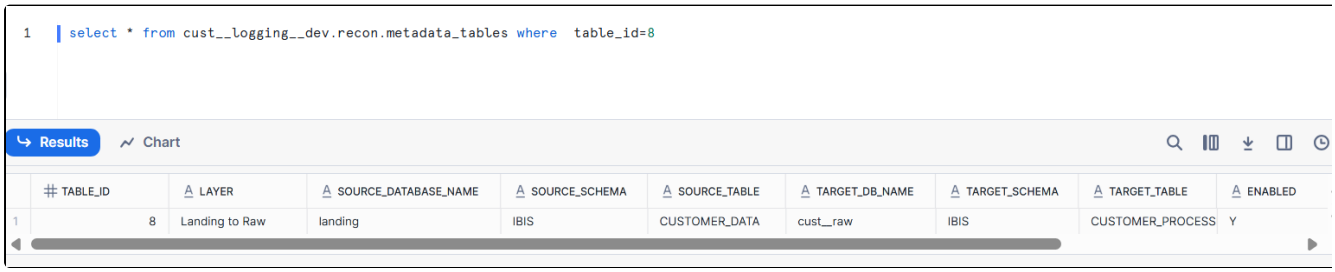
Test Case - Full Row Count Validation: Customer Data

Description This test validates that the number of active customer records matches between the source and target tables.

Step 1 : Configure data in Metadata_tables

Below are fields to be configure for metadata tables

select * from cust__logging__dev.recon.metadata_tables where table_id=8;



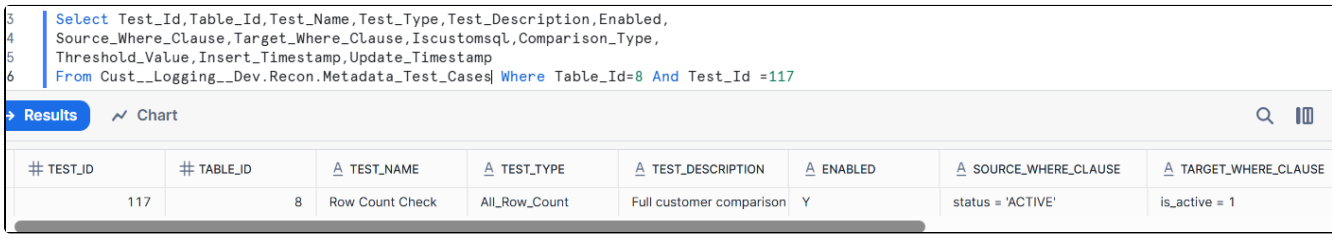
	TABLE_ID	LAYER	SOURCE_DATABASE_NAME	SOURCE_SCHEMA	SOURCE_TABLE	TARGET_DB_NAME	TARGET_SCHEMA	TARGET_TABLE	ENABLED
1	8	Landing to Raw	landing	IBIS	CUSTOMER_DATA	cust__raw	IBIS	CUSTOMER_PROCESS	Y

Step 2 :Configure data in Metadata_test_cases

Below are mandatory fields to be configure for the metadata test cases

SQL:

Select Test_Id,Table_Id,Test_Name,Test_Type,Test_Description,Enabled,
Source_Where_Clause,Target_Where_Clause,Iscustomsql,Comparison_Type,
Threshold_Value,Insert_Timestamp,Update_Timestamp
From Cust__Logging__Dev.Recon.Metadata_Test_Cases Where Table_Id=8 And Test_Id =117



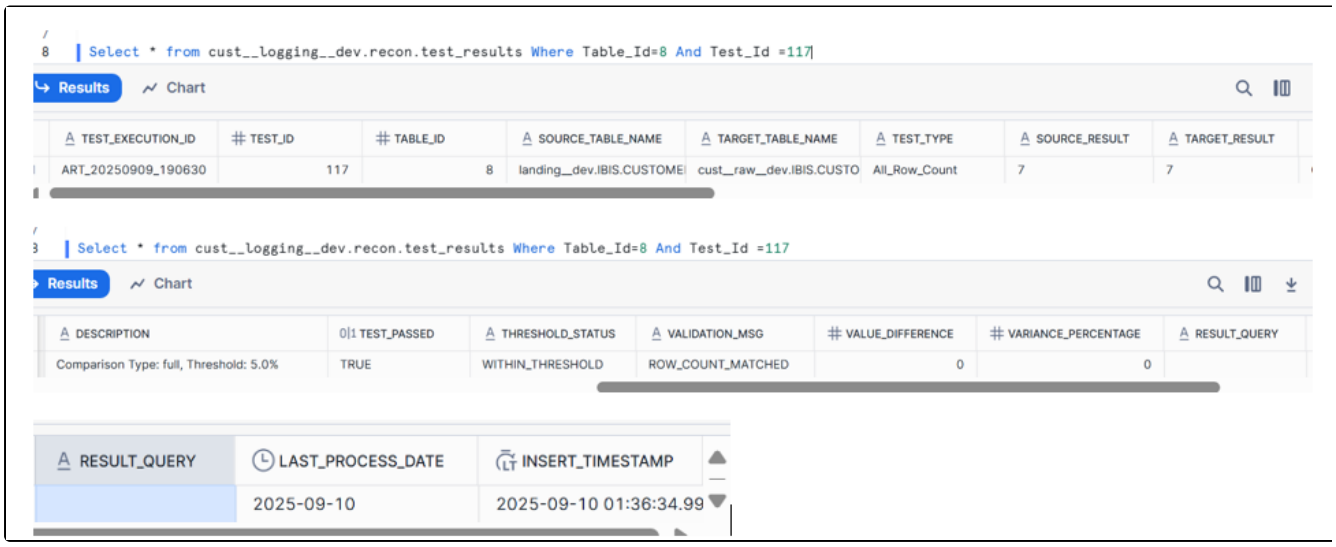
TEST_ID	TABLE_ID	TEST_NAME	TEST_TYPE	TEST_DESCRIPTION	ENABLED	SOURCE_WHERE_CLAUSE	TARGET_WHERE_CLAUSE
117	8	Row Count Check	All_Row_Count	Full customer comparison	Y	status = 'ACTIVE'	is_active = 1

ISCUSTOMSQL	COMPARISON_TYPE	THRESHOLD_VALUE	INSERT_TIMESTAMP	UPDATE_TIMESTAMP
N	full	5.0	2025-09-08 21:36:33.87	2025-09-08 21:36:33.87

Step 3: Execute the recon airflow dag and verify the results in final table

Below are the fields data will populate.

Select * from cust__logging__dev.recon.test_results Where Table_Id=8 And Test_Id =117;



Conclusion: Perfect match between source and target active customers.

Test Case - Custom SQL Row Count Validation: Customer Data

Description: This test uses custom SQL to compare active customer records between source and target, including a column mapping.

Step 1 : Configure data in Metadata_tables

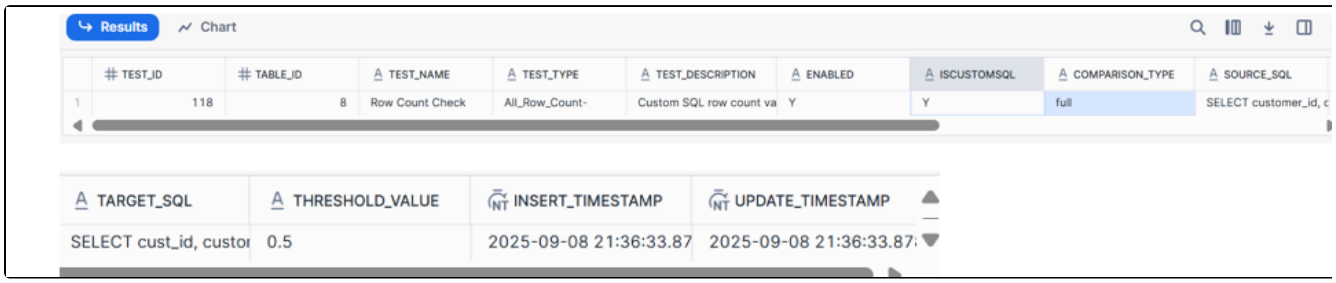
Below are fields to be configure for metadata tables

select * from cust__logging__dev.recon.metadata_tables where table_id=8;

Step 2 :Configure data in Metadata_test_cases

Below are mandatory fields to be configure for the metadata test cases

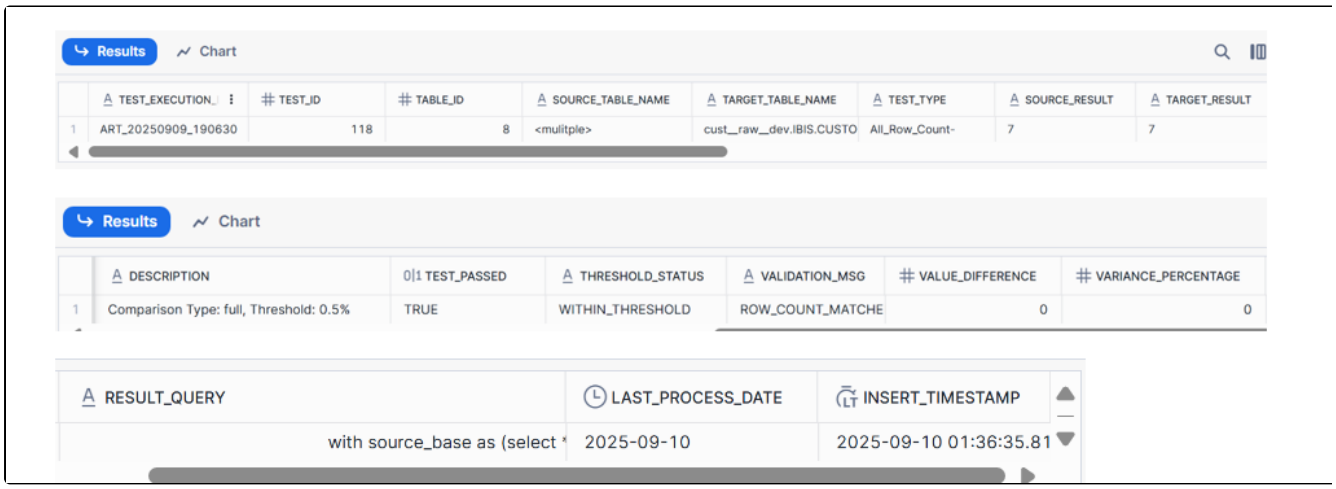
Select Test_Id,Table_Id,Test_Name,Test_Type,Test_Description,Enabled,Iscustomsql,Comparison_Type,Source_Sql,Target_Sql, Threshold_Value,Insert_Timestamp,Update_Timestamp
From Cust__Logging__Dev.Recon.Metadata_Test_Cases Where Table_Id=8 And Test_Id =118;



Step 3: Execute the recon airflow dag and verify the results in final table

Below are the fields data will populate

Select * from cust__logging__dev.recon.test_results Where Table_Id=8 And Test_Id =118;

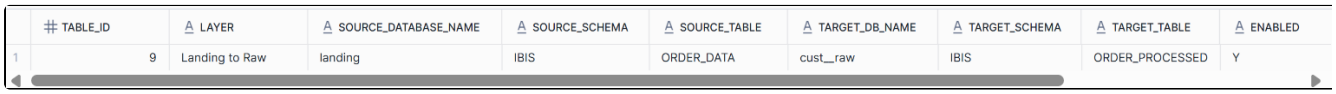


Conclusion: Custom SQL validation passed with perfect match

Test Case - Incremental Row Count Check: Order Data

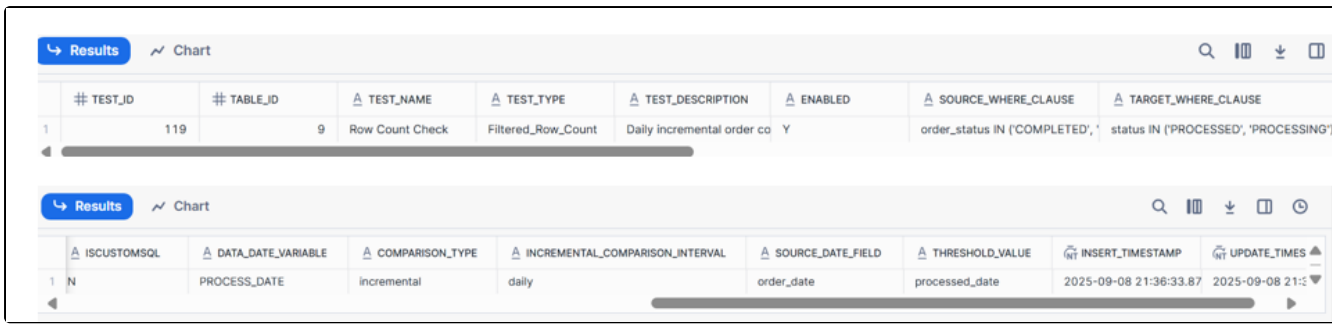
Description: This test checks daily incremental sync for order data with filters based on PROCESS_DATE = '2024-01-15'.

Step 1 : Configure data in Metadata_tables
Below are fields to be configure for metadata tables
select * from cust__logging__dev.recon.metadata_tables where table_id=9;



Step 2 :Configure data in Metadata_test_cases
Below are mandatory fields to be configure for the metadata test cases

Select Test_Id,Table_Id,Test_Name,Test_Type,Test_Description,Enabled,
Source_Where_Clause,Target_Where_Clause,Iscustomsql,Data_Date_Variable,
Comparison_Type,incremental_comparison_interval,source_date_field,target_date_field
Threshold_Value,Insert_Timestamp,Update_Timestamp
From Cust__Logging__Dev.Recon.Metadata_Test_Cases Where Table_Id=9 And Test_Id =119;




```
-- Explanation Of Compiled Elements:
-- 1. Data Date Processing:
--    - data_date_variable = 'PROCESS_DATE'
--    - var('PROCESS_DATE') = '2024-01-15'
--    - data_date_sql = TO_DATE('2024-01-15')
-- 2. Incremental Comparison Logic:
--    - incremental_date_filter_start = DATEADD(day, -1, TO_DATE('2024-01-15')) = '2024-01-14'
--    - incremental_date_filter_end = TO_DATE('2024-01-15')
--    - Looking at data from 2024-01-14 to 2024-01-15
-- 3. Where Clause :
--    - Source: order_status IN ('COMPLETED', 'PENDING', 'PROCESSING')
--    - Target: status IN ('PROCESSED', 'PROCESSING')
--    - Source: Additional date filters: order_date >= '2024-01-14' AND order_date < '2024-01-15'
    - Target: Additional date filters: processed_date >= '2024-01-14' AND processed_date < '2024-01-15'
-- 4. Threshold Comparison:
--    - threshold_value = 2.0%
--    - Actual variance: 50% (exceeds threshold significantly)
-- 5.  Explicit Date Columns:
--    - source_date_field: order_date
--    - target_date_field: processed_date
```

Step 3: Execute the recon airflow dag and verify the results in final table
Below are the fields data will populate

Select * from cust__logging__dev.recon.test_results Where Table_Id=9 And Test_Id =119;

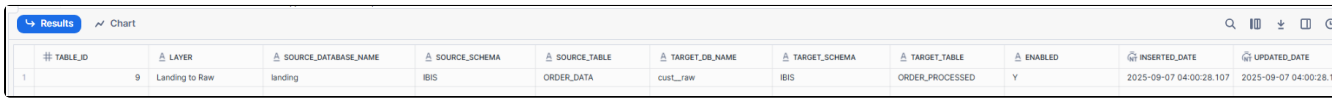
Conclusion: Source has 1 extra record, indicating potential data issue

Test Case - Incremental Row Count with Explicit Date Filter: Order Data

Description: This test checks Daily incremental row count check for orders based on filter condition .

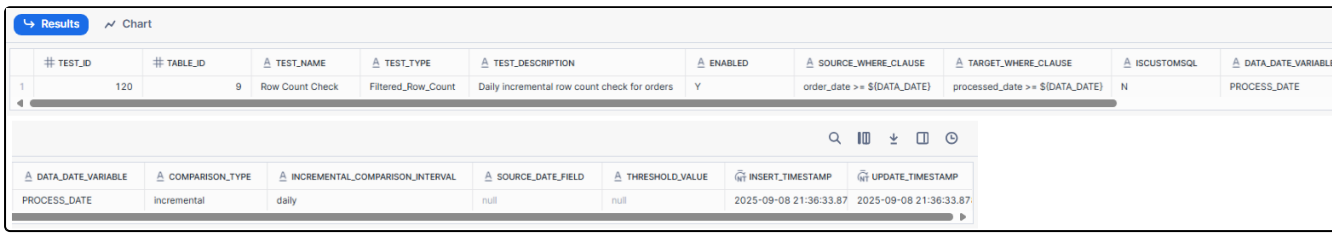
Step 1 : Configure data in Metadata_tables
Below are fields to be configure for metadata tables

SELECT * FROM CUST__LOGGING__DEV.RECON.METADATA_TABLES WHERE TABLE_ID=9;



Step 2 :Configure data in Metadata_test_cases
Below are mandatory fields to be configure for the metadata test cases

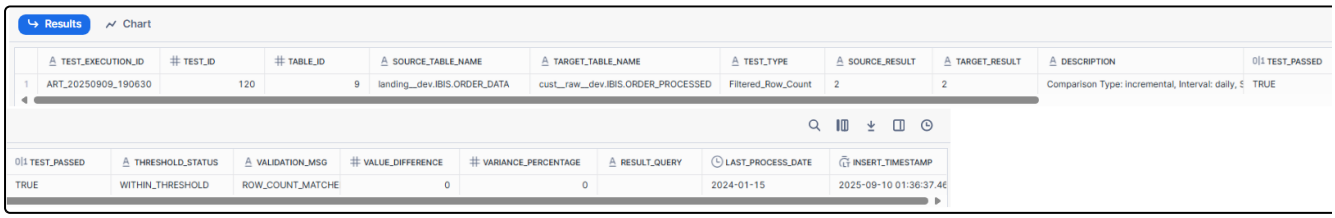
SELECT TEST_ID,TABLE_ID,TEST_NAME,TEST_TYPE,TEST_DESCRIPTION,ENABLED,SOURCE_WHERE_CLAUSE,
TARGET_WHERE_CLAUSE,ISCUSTOMSQL,DATA_DATE_VARIABLE,
COMPARISON_TYPE,INCREMENTAL_COMPARISON_INTERVAL,SOURCE_DATE_FIELD,TARGET_DATE_FIELDTHRESHOLD_VALUE,
INSERT_TIMESTAMP,UPDATE_TIMESTAMP
FROM CUST__LOGGING__DEV.RECON.METADATA_TEST_CASES WHERE TABLE_ID=9 AND TEST_ID =120;




```
-- Explanation Of Compiled Elements:
-- 1. Data Date Processing:
--     - data_date_variable = 'PROCESS_DATE'
--     - var('PROCESS_DATE') = '2024-01-15'
--     - data_date_sql = TO_DATE('2024-01-15')
-- 2. Where Clause Substitution:
--     - Source WHERE: order_date >= ${DATA_DATE}  order_date >= '2024-01-15'
--     - Target WHERE: processed_date >= ${DATA_DATE}  processed_date >= '2024-01-15'
-- 3. Incremental logic :
---      - Looking at data greater than 2024-01-15
-- 4. No Explicit Date Columns:
--     - source_date_field: None (uses default)
--     - target_date_field: None (uses default)
```

Step 3: Execute the recon airflow dag and verify the results in final table
Below are the fields data will populate

SELECT * FROM CUST__LOGGING__DEV.RECON.TEST_RESULTS WHERE TABLE_ID=9 AND TEST_ID =120;



Conclusion: Perfect match when using explicit date filters in WHERE clause

Test Case - Grouped Row Count by Category: Product Data

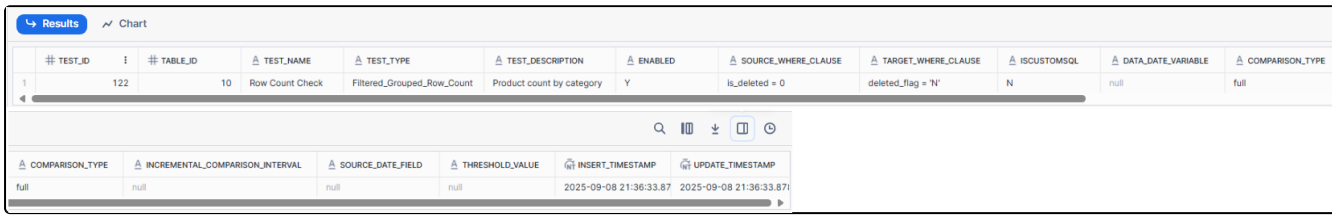
Description: This test checks Product count by category .

Step 1 : Configure data in Metadata_tables
Below are fields to be configure for metadata tables

SELECT * FROM CUST__LOGGING__DEV.RECON.METADATA_TABLES WHERE TABLE_ID=10;

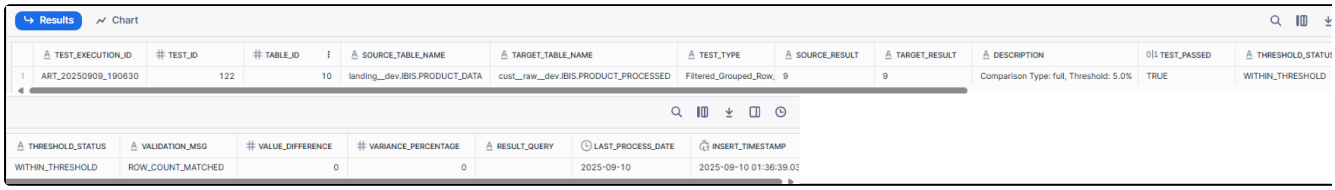
Step 2 :Configure data in Metadata_test_cases
Below are mandatory fields to be configure for the metadata test cases

SELECT TEST_ID,TABLE_ID,TEST_NAME,TEST_TYPE,TEST_DESCRIPTION,ENABLED,SOURCE_WHERE_CLAUSE,
TARGET_WHERE_CLAUSE,ISCUSTOMSQL,DATA_DATE_VARIABLE,
COMPARISON_TYPE,INCREMENTAL_COMPARISON_INTERVAL,SOURCE_DATE_FIELD,TARGET_DATE_FIELDTHRESHOLD_VALUE,
INSERT_TIMESTAMP,UPDATE_TIMESTAMP
FROM CUST__LOGGING__DEV.RECON.METADATA_TEST_CASES WHERE TABLE_ID=10 AND TEST_ID =122;



Step 3: Execute the recon airflow dag and verify the results in final table
Below are the fields data will populate

SELECT * FROM CUST__LOGGING__DEV.RECON.TEST_RESULTS WHERE TABLE_ID=10 AND TEST_ID =122;



Conclusion: Perfect match when using explicit date filters in WHERE clause

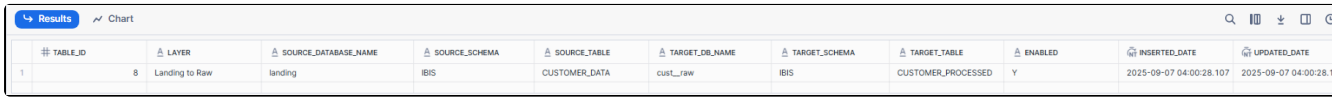
Test Case - Data Freshness Check: Customer Data

Description: This test checks the Customer data freshness validation - 12 hour threshold

Step 1 : Configure data in Metadata_tables

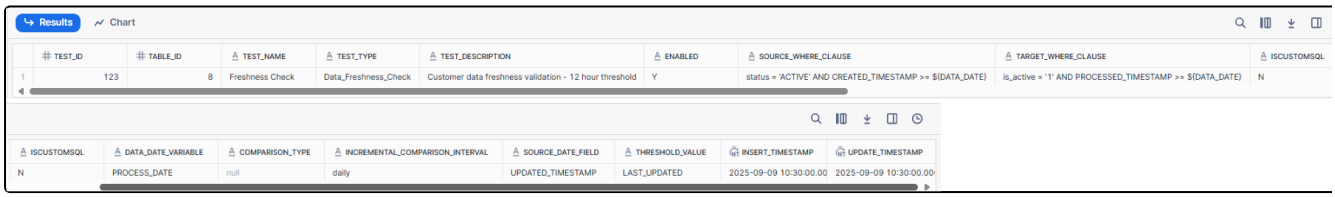
Below are fields to be configure for metadata tables

SELECT * FROM CUST__LOGGING__DEV.RECON.METADATA_TABLES WHERE TABLE_ID=8;



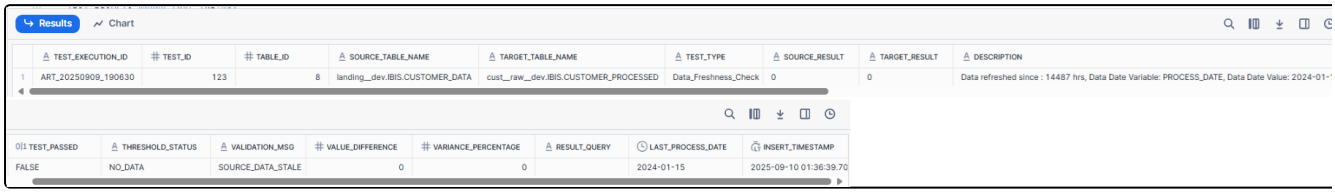
Step 2 :Configure data in Metadata_test_cases
Below are mandatory fields to be configure for the metadata test cases

SELECT TEST_ID,TABLE_ID,TEST_NAME,TEST_TYPE,TEST_DESCRIPTION,ENABLED,SOURCE_WHERE_CLAUSE,
TARGET_WHERE_CLAUSE,ISCUSTOMSQL,DATA_DATE_VARIABLE,
COMPARISON_TYPE,INCREMENTAL_COMPARISON_INTERVAL,SOURCE_DATE_FIELD,TARGET_DATE_FIELDTHRESHOLD_VALUE,
INSERT_TIMESTAMP,UPDATE_TIMESTAMP
FROM CUST__LOGGING__DEV.RECON.METADATA_TEST_CASES WHERE TABLE_ID=8 AND TEST_ID =123;



Step 3: Execute the recon airflow dag and verify the results in final table
Below are the fields data will populate

SELECT * FROM CUST__LOGGING__DEV.RECON.TEST_RESULTS WHERE TABLE_ID=8 AND TEST_ID =123;



Conclusion: Data is severely stale (603+ days old), needs immediate refresh

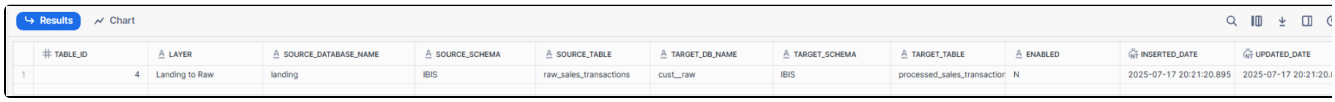
Test Case - Aggregation Check: product_category

Description: This test checks the aggregation check and Calualate Total_Sales_Amount & Quantity _By_Product_Category

Step 1 : Configure data in Metadata_tables

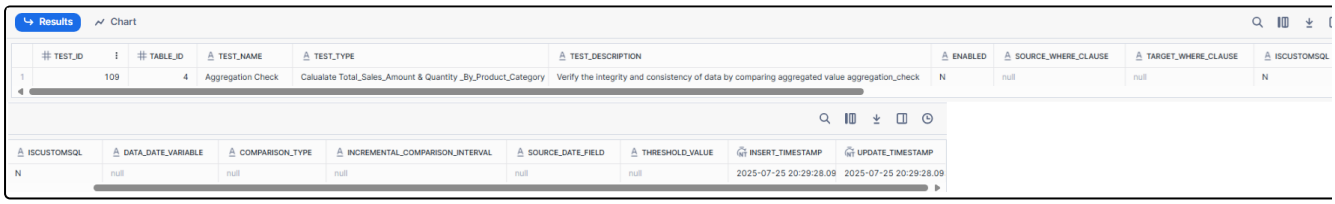
Below are fields to be configure for metadata tables

SELECT * FROM CUST__LOGGING__DEV.RECON.METADATA_TABLES WHERE TABLE_ID=4;



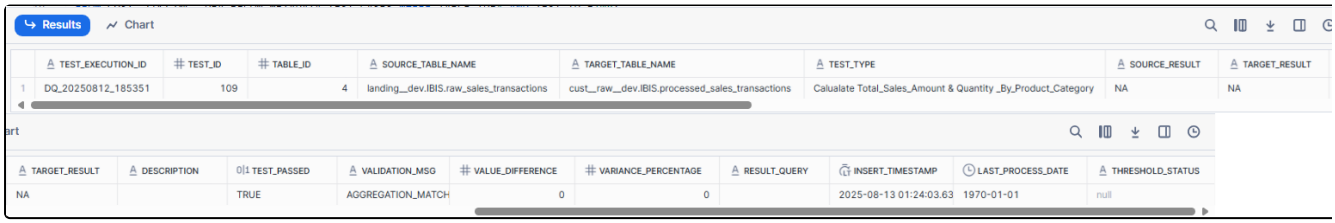
Step 2 :Configure data in Metadata_test_cases
Below are mandatory fields to be configure for the metadata test cases

SELECT TEST_ID,TABLE_ID,TEST_NAME,TEST_TYPE,TEST_DESCRIPTION,ENABLED,SOURCE_WHERE_CLAUSE,
TARGET_WHERE_CLAUSE,ISCUSTOMSQL,DATA_DATE_VARIABLE,
COMPARISON_TYPE,INCREMENTAL_COMPARISON_INTERVAL,SOURCE_DATE_FIELD,TARGET_DATE_FIELDTHRESHOLD_VALUE,
INSERT_TIMESTAMP,UPDATE_TIMESTAMP
FROM CUST__LOGGING__DEV.RECON.METADATA_TEST_CASES WHERE TABLE_ID=4 AND TEST_ID =109;



Step 3: Execute the recon airflow dag and verify the results in final table
Below are the fields data will populate

SELECT * FROM CUST__LOGGING__DEV.RECON.TEST_RESULTS WHERE TABLE_ID=4 AND TEST_ID =109;



Conclusion: Perfect match in category-wise product aggerate total.

Team User Guide for Reconciliation and Testing

This following link further explain step by step instructions on how seed files are used to accelerate the onboarding of the required metadata tables

Link:- DBT Recon Framework– Metadata & Seed Onboarding Process

Conclusion

This approach empowers data teams to detect and resolve data quality issues proactively while maintaining high levels of performance and scalability. The framework's modular design ensures that reconciliation processes can evolve with changing business requirements while preserving data integrity standards across the entire ecosystem.

By centralizing reconciliation logic in reusable dbt macros and orchestrating execution through Airflow workflows, organizations achieve both operational efficiency and data reliability at enterprise scale.

FAQ

Section to capture general clarifications that were raised and responses, to help understand the framework better.

No.	Question	Answer / Clarification
1	What is the purpose of the Reconciliation Framework?	The framework provides an automated, metadata-driven approach to validate data consistency across systems. It uses dbt macros and Airflow orchestration to ensure data accuracy, completeness, and reliability.
2	Is this framework meant to replace existing dbt tests?	**No**. It is complementary to dbt tests. dbt validates data within models, whereas the reconciliation framework validates cross-system or cross-batch consistency (e.g., comparing aggregates between source and target).
3	Where does the reconciliation logic run — inside dbt or in a separate DAG?	Separate Airflow DAG after load completed.
4	How are thresholds defined?	Thresholds are defined at the **test-case level** in `METADATA_TEST_CASES` using:
5	What does `VARIANCE_PERCENTAGE` measure — source vs target, or across runs?	`VARIANCE_PERCENTAGE` represents the percentage difference in record counts between source and target datasets within the same run. It is calculated using the formula: • If the greatest value between `source total records` and `target total records` is equal to zero, then `VARIANCE_PERCENTAGE` is set to 0. • Else, it is calculated as: (Absolute difference between source and target total records × 100) ÷ Greatest of source or target total records. • Finally, the result is rounded to 2 decimal places
6	Can we capture `DATA_DATE` for reporting?	**Yes**. A `DATA_DATE` field is available in `TEST_RESULTS` to capture the RUN_DATE Value (e.g : 2025-10-07)
7	How do we pass date or parameter values dynamically in SQL?	Dynamic parameters such as `@DATA_DATE` can be used, **e.g 1** `WHERE POSTING_DATE = @DATA_DATE` **e.g 2** `WHERE @DATA_DATE BETWEEN DWH_EFFECTIVE_FROM_TSTAMP AND DWH_EFFECTIVE_TO_TSTAMP` These are substituted dynamically during execution.
8	What happens when a reconciliation test fails?	When a test fails: • A failure record is logged in `TEST_RESULTS` with reason and timestamps. • Users can perform **Analysis** using query results in **Test_results** table
9	How can users review reconciliation outcomes?	Users can query `TEST_RESULTS` to view discrepancies.
10	Are screenshots and examples included?	**Yes**. The document includes: • Screenshots of metadata and result tables. • Practical examples showing how `METADATA_TEST_CASES` relate to `TEST_RESULTS`.
11	What about access control and security?	Additional **Access Control and RBAC policies** are required to restrict access to metadata and result tables, ensuring only authorized users can configure or review test outcomes.
12	Are mandatory columns defined in the framework tables?	**Yes**. Each metadata and result table (e.g., `METADATA_TEST_CASES`, `TEST_RESULTS`) has a defined set of **mandatory columns** such as `TEST_CASE_ID`, `TEST_NAME`, `SOURCE_QUERY`, `TARGET_QUERY` ensure data completeness.
13	How do we know if a test case is enabled or disabled?	Each test case in `METADATA_TEST_CASES` includes an **`IS_ENABLED`** flag. Only records with `IS_ENABLED = Y` are executed during reconciliation runs, allowing selective activation or deactivation of tests.

14	Is Kafka in scope for reconciliation?	**No.** Kafka source data and **Fivetran-based data reconciliation are out of scope** for this framework. The current scope covers **database-to-database** within the Snowflake and dbt ecosystem.
15	Can we get any notification in case any test case failed?	**No.** There is currently no automated notification mechanism. Users must review the `TEST_RESULTS` table — specifically the `TEST_PASSED` column (where value = `FALSE`) — to identify failed tests and analyze the corresponding query results manually.
16	Can we perform the reconciliation check using Custom SQL?	**Yes.** Custom SQL-based reconciliation is supported. Set the `IS_CUSTOM_SQL` flag to `'Y'` in the `METADATA_TEST_CASES` table, and provide your SQL logic in the `SOURCE_SQL` and `TARGET_SQL` fields to override default query logic.

Review Comments & Responses

Reviewer	Comment / Suggestion	Response / Action Taken	Status	Date
Yi-Jing Chung	would we add reference to data controls this detail design is for?	Included the confluence page links	Resolved	09-Sep-2025
Yi-Jing Chung	How come Kafka source is not covered her	Explained and note added in scope section	Resolved	09-Sep-2025
Dandan Cai	Defect Analysis	Explained and note and in scope section	Resolved	10-Sep-2025
Yi-Jing Chung	not 100% sure this should be called DQ? would it be confusing with the current DQ Informatica tool?	Modified code as per the comment suggested	Resolved	10-Sep-2025
Yi-Jing Chung	his is an automated, metadata	Provided the response	Resolved	10-Sep-2025